

Algoritmi Randomizzati e di Approssimazione

Rilassamento LP, arrotondamento randomizzato e derandomizzazione

Appunti rielaborati dalle slide del corso *Algorithm Design*
S. Leonardi, R. Reiffenhäuser

Panoramica del documento

Questo documento riprende in italiano, in forma discorsiva ed estesa, le tre parti della lezione:

1. **Weighted Max-SAT**: formulazione come programma lineare (LP) e arrotondamento randomizzato per ottenere algoritmi di approssimazione (2-approssimazione, $e/(e-1)$ -approssimazione, $4/3$ -approssimazione).
2. **Max-Weighted-Cut**: algoritmo randomizzato 2-approssimato, sua *derandomizzazione* tramite il metodo delle aspettative condizionate, e un algoritmo greedy deterministico equivalente.
3. Un esercizio guidato sul problema del **Multiway Cut**.

1 Il problema Weighted Max-SAT

Definizione del problema

Siamo dati:

- una collezione di clausole $C = \{C_1, \dots, C_t\}$ in forma disgiuntiva (OR di letterali);
- un peso $w_i \geq 0$ per ogni clausola C_i ;
- n variabili booleane $x_1, \dots, x_n$, dove ogni letterale di C_i è x_j oppure $\overline{x_j}$.

Obiettivo: trovare un'assegnazione $x_1, \dots, x_n \in \{0, 1\}$ che *massimizzi la somma dei pesi delle clausole soddisfatte*.

Nelle lezioni precedenti il problema era stato affrontato con la tecnica classica: scrivere un LP che rilassa il problema intero, risolverlo in tempo polinomiale, e poi *arrotondare* la soluzione frazionaria a una soluzione intera (ammissibile) che approssima l'ottimo. Qui la novità è che il passo di arrotondamento è **randomizzato**: si assegna ogni variabile a 1 o a 0 con una certa probabilità, invece che con una regola deterministica.

Il valore restituito dall'algoritmo diventa allora una *variabile aleatoria*, e si dimostrano garanzie sul suo **valore atteso**:

$$\mathbb{E}[\text{ALG}(I)] \geq \frac{1}{2} \text{OPT}(I) \quad \forall \text{istanza } I.$$

Un punto importante discusso a lezione è che un algoritmo randomizzato con questa garanzia *in valore atteso* può quasi sempre essere trasformato in un algoritmo deterministico con la stessa garanzia: questo procedimento si chiama **derandomizzazione** (lo vedremo in dettaglio nella Parte 2, per Max-Cut).

1.1 Algoritmo 1: arrotondamento a moneta equa (Johnson, 1974)

Algoritmo

Per ogni $j = 1, \dots, n$: poni $x_j = 1$ con probabilità $\frac{1}{2}$, e $x_j = 0$ con probabilità $\frac{1}{2}$ (indipendentemente).

Analisi. Sia k_i il numero di letterali nella clausola C_i . La clausola C_i *non* è soddisfatta solo se tutti i suoi k_i letterali risultano falsi, evento che (per indipendenza) ha probabilità $(1/2)^{k_i}$. Quindi

$$\Pr[C_i \text{ soddisfatta}] \geq 1 - \left(\frac{1}{2}\right)^{k_i} \geq \frac{1}{2} \quad (\text{poiché } k_i \geq 1).$$

Per linearità del valore atteso,

$$\mathbb{E}[\text{ALG}] = \sum_{i=1}^t w_i \Pr[C_i \text{ soddisfatta}] \geq \frac{1}{2} \sum_{i=1}^t w_i \geq \frac{1}{2} \text{OPT},$$

dove l'ultima disuguaglianza vale perché la somma di *tutti* i pesi è banalmente un limite superiore per l'ottimo (nel caso migliore si soddisfano tutte le clausole).

Esempio

Clausola $C_1 = x_1 \vee \overline{x_2}$ (quindi $k_1 = 2$). Con moneta equa,

$$\Pr[C_1 \text{ falsa}] = \Pr[x_1 = 0] \cdot \Pr[x_2 = 1] = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4},$$

quindi $\Pr[C_1 \text{ vera}] = 3/4 \geq 1 - (1/2)^2 = 3/4$: il limite è raggiunto con uguaglianza.

Un vantaggio pratico di questo algoritmo è che si può **derandomizzare facilmente** (cioè renderlo deterministico mantenendo la garanzia), perché la scelta di ogni x_j non dipende dalle altre.

1.2 Un rapporto migliore: l'algoritmo di Yannakakis (1992)

Trattando in modo differenziato le clausole con un solo letterale (che con la moneta equa vengono soddisfatte solo con probabilità $1/2$, il caso peggiore), Yannakakis ottiene un rapporto di approssimazione migliore, pari a **4/3**. L'idea di fondo — combinare più strategie per coprire i casi in cui una singola strategia è debole — è quella che ritroveremo più sotto nell'algoritmo di Goemans–Williamson combinato con Johnson.

1.3 Algoritmo 2: arrotondamento basato su LP (Goemans–Williamson, 1993)

L'idea è di non lanciare più una moneta equa per ogni variabile, ma di **prima risolvere il rilassamento LP** del problema, ottenendo per ogni variabile un valore frazionario $x_j^* \in [0, 1]$ interpretabile come una probabilità, e poi arrotondare in modo randomizzato *secondo quella probabilità*.

Formulazione come programma lineare

Per ogni clausola C_i , siano:

$$T_i = \{j : x_j \text{ compare non negata in } C_i\}, \quad F_i = \{j : x_j \text{ compare negata in } C_i\},$$

e sia $z_i \in \{0, 1\}$ la variabile indicatrice “ C_i è soddisfatta”. Il problema intero è

$$\max \sum_{i=1}^t z_i w_i \quad \text{s.v.} \quad \sum_{j \in T_i} x_j + \sum_{j \in F_i} (1 - x_j) \geq z_i, \quad x_j, z_i \in \{0, 1\}.$$

Il rilassamento LP si ottiene sostituendo $x_j, z_i \in \{0, 1\}$ con $x_j, z_i \in [0, 1]$ (in pratica basta imporre $x_j \geq 0$ e $z_i \leq 1$).

Schema di arrotondamento

Sia (x^*, z^*) la soluzione ottima del rilassamento. Per ogni j :

$$x_j = 1 \text{ con probabilità } x_j^*, \quad x_j = 0 \text{ con probabilità } 1 - x_j^*.$$

Lemma centrale

Con questo arrotondamento,

$$\Pr[C_i \text{ soddisfatta}] \geq 1 - \prod_{j \in T_i} (1 - x_j^*) \prod_{j \in F_i} x_j^* \geq \alpha_{k_i} z_i^*, \quad \text{con } \alpha_k = 1 - \left(1 - \frac{1}{k}\right)^k.$$

Idea della dimostrazione. Per semplicità si assume (senza perdita di generalità, sostituendo eventualmente x_j con $1 - x_j$) che tutti i letterali della clausola siano non negati, così che $C_i = x_{i_1} \vee \dots \vee x_{i_{k_i}}$. Allora

$$\Pr[C_i \text{ soddisfatta}] = 1 - \prod_{j=1}^{k_i} (1 - x_j^*).$$

Applicando la disuguaglianza delle medie (media geometrica $\leq$ media aritmetica) al prodotto, e poi usando il vincolo LP $\sum_j x_j^* \geq z_i^*$, si ottiene

$$1 - \prod_{j=1}^{k_i} (1 - x_j^*) \geq 1 - \left(1 - \frac{\sum_j x_j^*}{k_i}\right)^{k_i} \geq 1 - \left(1 - \frac{z_i^*}{k_i}\right)^{k_i} \geq \alpha_{k_i} z_i^*,$$

dove l'ultimo passaggio segue dalla concavità della funzione $z \mapsto 1 - (1 - z/k)^k$ su $[0, 1]$ (vale con uguaglianza per $z = 1$, dando proprio α_k , e la funzione sta sopra la corda che congiunge $(0, 0)$ con $(1, \alpha_k)$).

Esempio

Per $k = 1$ (clausola con un solo letterale): $\alpha_1 = 1 - (1 - 1)^1 = 1$. Coerente: se $z_i^* = 1$ e $k_i = 1$ allora x_j^* deve valere 1, e la clausola è soddisfatta con probabilità esattamente 1.

Per $k = 2$: $\alpha_2 = 1 - (1/2)^2 = 3/4$.

Per $k \rightarrow \infty$: $\alpha_k \rightarrow 1 - 1/e \approx 0.632$.

Rapporto di approssimazione $e/(e-1)$

Poiché $(1 - \frac{1}{k})^k \leq \frac{1}{e}$ per ogni $k \geq 1$, si ha $\alpha_k \geq 1 - \frac{1}{e}$ per ogni k , dunque

$$\mathbb{E}[\text{ALG}] \geq \sum_{i=1}^t \alpha_{k_i} w_i z_i^* \geq \left(1 - \frac{1}{e}\right) \sum_{i=1}^t w_i z_i^* \geq \left(1 - \frac{1}{e}\right) \text{OPT}_{LP} \geq \frac{e-1}{e} \text{OPT},$$

cioè un'approssimazione con rapporto ≈ 1.582 ($\frac{e}{e-1}$). Si noti che qui il limite superiore usato per l'ottimo non è più la banale somma dei pesi, ma il valore ottimo del *rilassamento LP*, che è sempre $\geq \text{OPT}$ intero, e quindi un limite più stretto e più informativo.

1.4 Combinare i due algoritmi: la 4/3-approssimazione

Confrontando i due bound trovati:

$$\text{Johnson: } \mathbb{E}[\text{ALG}] \geq \sum_i \left(1 - \frac{1}{2^{k_i}}\right) w_i z_i^*, \quad \text{G\&W: } \mathbb{E}[\text{ALG}] \geq \sum_i \alpha_{k_i} w_i z_i^*.$$

Osservazione

Johnson è forte quando k_i è **grande** (il fattore $1 - 2^{-k_i}$ tende rapidamente a 1), mentre Goemans–Williamson è forte quando k_i è **piccolo** (per $k_i = 1, 2$ dà il valore massimo possibile o quasi). I due algoritmi hanno quindi “casi cattivi” complementari.

Strategia: lanciare una moneta equa per scegliere quale dei due algoritmi eseguire: con probabilità $\frac{1}{2}$ Johnson, con probabilità $\frac{1}{2}$ Goemans–Williamson.

4/3-approssimazione

$$\Pr[C_i \text{ soddisfatta}] \geq \frac{1}{2} \left[(1 - 2^{-k_i}) + \alpha_{k_i} \right] z_i^* \geq \frac{3}{4} z_i^*,$$

da cui

$$\mathbb{E}[\text{ALG}] \geq \frac{3}{4} \sum_{i=1}^t w_i z_i^* \geq \frac{3}{4} \text{OPT}.$$

Il rapporto di approssimazione è dunque $4/3$.

Il punto cruciale (visibile anche dal grafico originale delle slide) è che, al variare di k , la media delle due curve $\frac{1}{2}(1 - 2^{-k})$ e $\frac{1}{2}\alpha_k$ non scende mai sotto $3/4$: per $k = 1$ Johnson è debole ($1/2$) ma G&W è perfetto ($\alpha_1 = 1$), e viceversa per k grande. La combinazione “spalma” il rischio.

2 Il problema Max-Weighted-Cut**Definizione del problema**

Dato un grafo $G = (V, E)$ e pesi $w : E \rightarrow \mathbb{R}^+$, trovare una partizione $(S, \bar{S})$ di V che massimizzi il peso totale degli archi *tagliati*, cioè con un estremo in S e l'altro in $\bar{S}$:

$$\sum_{(i,j) \in E : i \in S, j \in \bar{S}} w_{(i,j)}.$$

2.1 Algoritmo randomizzato base: 2-approssimazione**Algoritmo**

Per ogni vertice $i \in V$, indipendentemente: assegna i a S con probabilità $\frac{1}{2}$, a $\bar{S}$ con probabilità $\frac{1}{2}$.

Per ogni arco $(i, j) \in E$, esso appartiene al taglio se e solo se i e j finiscono in parti diverse. Poiché le assegnazioni sono indipendenti e uniformi,

$$\Pr[(i \in S \wedge j \in \bar{S}) \vee (i \in \bar{S} \wedge j \in S)] = \frac{1}{2}.$$

Per linearità del valore atteso,

$$\mathbb{E}[\text{ALG}] = \frac{1}{2} \sum_{(i,j) \in E} w_{(i,j)} \geq \frac{1}{2} \text{OPT},$$

dove, come per Max-SAT, si è usato il limite superiore banale $\text{OPT} \leq \sum_{(i,j) \in E} w_{(i,j)}$ (il taglio massimo non può contenere più della somma di tutti i pesi degli archi).

Esempio

Grafo con 3 vertici $\{a, b, c\}$ e archi $(a, b) = 1$, $(b, c) = 1$, $(a, c) = 1$ (triangolo, pesi unitari). $\text{OPT} = 2$ (qualunque bipartizione non banale taglia esattamente 2 archi su 3, essendo il grafo un ciclo dispari che non è bipartito). Il valore atteso dell'algoritmo randomizzato è $\frac{1}{2} \cdot 3 = 1.5$, che rispetta $\mathbb{E}[\text{ALG}] \geq \frac{1}{2}\text{OPT} = 1$.

Nota

Usando invece un limite superiore più raffinato per OPT ottenuto tramite *programmazione quadratica* (SDP), Goemans e Williamson (1994) hanno dimostrato un'approssimazione molto migliore, ≈ 0.8785 (celebre risultato basato sul rilassamento a vettori sulla sfera).

2.2 Derandomizzazione tramite aspettative condizionate

L'obiettivo è ora trasformare l'algoritmo randomizzato in uno **deterministico** che mantenga lo stesso rapporto di approssimazione (2).

Definiamo $x_i = 1$ se $v_i \in A$ e $x_i = 0$ se $v_i \in B$ (usiamo A, B al posto di $S, \bar{S}$ per seguire la notazione della lezione), e sia W il peso del taglio indotto. L'idea è **gradualmente eliminare la casualità**, variabile per variabile, mantenendo sempre il valore atteso più alto possibile.

Strategia

Fissate le assegnazioni di $x_1, \dots, x_i$, si sceglie per x_{i+1} il valore che **massimizza** il valore atteso del taglio, supponendo che tutte le variabili successive siano ancora scelte a caso.

Per la legge delle aspettative totali (condizionando sull'ultimo bit ancora casuale):

$$\underbrace{\mathbb{E}[W \mid x_1=a_1, \dots, x_i=a_i]}_{(1)} = \frac{1}{2} \underbrace{\mathbb{E}[W \mid x_1=a_1, \dots, x_i=a_i, x_{i+1}=1]}_{(2)} + \frac{1}{2} \underbrace{\mathbb{E}[W \mid x_1=a_1, \dots, x_i=a_i, x_{i+1}=0]}_{(3)}.$$

Due osservazioni fanno funzionare il metodo:

1. **(2) $\geq$ (3)** se e solo se $w(v_{i+1}, B) \geq w(v_{i+1}, A)$ (conviene mettere v_{i+1} dalla parte opposta rispetto a dove ha più peso di archi incidenti già assegnato — assegnarlo ad A “salva” il peso verso B).
2. Poiché (1) è la *media* di (2) e (3), necessariamente $(1) \leq \max\{(2), (3)\}$: **non tutte le scelte per x_{i+1} possono essere peggiori di una scelta casuale**. Scegliendo sempre l'opzione migliore, il valore atteso non può mai diminuire rispetto allo stadio precedente.

Algoritmo deterministico

$A = \{v_1\}, \quad B = \emptyset.$

for $i = 2, \dots, n$:

Aggiungi v_i all'insieme che massimizza il valore atteso condizionato del taglio.

Poiché ogni passo non peggiora mai il valore atteso rispetto al passo precedente, e si parte da $\mathbb{E}[W] \geq \frac{1}{2}\text{OPT}$ (garanzia dell'algoritmo randomizzato), l'algoritmo deterministico finale ha peso $\geq \frac{1}{2}\text{OPT}$: **si mantiene il rapporto 2**. Questo procedimento si chiama *derandomizzazione tramite aspettative condizionate* (*method of conditional expectations*).

Efficienza: come si calcola il passo?

Al passo $i + 1$, condizionando sulle scelte già fissate $x_1, \dots, x_i$ (denotiamo con $W_{A \wedge B}$ il peso già certamente tagliato tra A e B fin qui):

$$\mathbb{E}[W \mid \dots, x_{i+1} = 1] = W_{A \wedge B} + w(v_{i+1}, A) + \frac{1}{2} \sum_{\substack{(v_j, v_k) \in E \\ j > i+1}} w(v_j, v_k),$$

$$\mathbb{E}[W \mid \dots, x_{i+1} = 0] = W_{A \wedge B} + w(v_{i+1}, B) + \frac{1}{2} \sum_{\substack{(v_j, v_k) \in E \\ j > i+1}} w(v_j, v_k).$$

Le due espressioni differiscono **solo** nel termine $w(v_{i+1}, A)$ oppure $w(v_{i+1}, B)$ (il peso da v_{i+1} verso i vertici già assegnati a A , risp. a B): tutto il resto è identico. Quindi **basta confrontare questi due numeri**, calcolabili in tempo polinomiale (basta sommare i pesi degli archi da v_{i+1} verso i vertici già collocati), senza dover ricalcolare l'intera aspettativa da zero ad ogni passo.

2.3 Algoritmo greedy equivalente per Max-Cut

La derandomizzazione porta naturalmente a un semplice algoritmo **greedy**, che è esattamente la traduzione operativa del ragionamento precedente.

Algoritmo greedy

Per $A \subseteq V$, sia $w(v, A)$ il peso totale degli archi tra v e i vertici di A .

```

 $A = \{v_1\}, \quad B = \{v_2\}.$ 
for  $v \in V \setminus \{v_1, v_2\}$  :
    if  $w(v, A) \geq w(v, B)$  then  $B \leftarrow B \cup \{v\}$ 
    else  $A \leftarrow A \cup \{v\}$ 

```

Restituisci (A, B) come $(S, \bar{S})$.

Il greedy è una 2-approssimazione

I vertici sono considerati nell'ordine $v_1, \dots, v_n$; l'arco (v_1, v_2) è sempre nel taglio. Sia $w_{<}(v_i)$ il peso totale degli archi da v_i verso $v_1, \dots, v_{i-1}$ (i vertici già collocati quando v_i viene processato). Poiché al momento di collocare v_i lo si mette dalla parte a cui è *meno* collegato (regola greedy), **almeno metà** del peso $w_{<}(v_i)$ finisce nel taglio:

$$\text{peso tagliato da } v_i \text{ verso i precedenti} \geq \frac{w_{<}(v_i)}{2}.$$

Sommando su tutti i vertici (e osservando che ogni arco (v_j, v_k) con $j < k$ contribuisce esattamente una volta a $w_{<}(v_k)$, quindi $\sum_i w_{<}(v_i) = \sum_{(i,j) \in E} w(i,j) \geq \text{OPT}$):

$$\text{ALG} \geq \frac{1}{2} \sum_{i=2}^n w_{<}(v_i) \geq \frac{\text{OPT}}{2}.$$

Esempio

Vertice v_i con archi verso i vertici già collocati di peso totale $w_{<}(v_i) = 1 + 2 + 4 = 7$ verso A e $3 + 5 = 8$ verso B (come nello schizzo delle slide). Poiché $8 > 7$, il greedy assegna v_i ad A : il peso "salvato" nel taglio è 8 (tutto ciò che va verso B), che è $\geq 7/2 = 3.5$. In generale il taglio ottenuto localmente è sempre almeno metà del massimo tra i due gruppi, quindi almeno metà

del totale $w_<(v_i)$.

Confronto con la derandomizzazione

Si noti che questo algoritmo greedy è *diverso ma strettamente imparentato* con l'algoritmo derandomizzato tramite aspettative condizionate: entrambi processano i vertici uno alla volta e scelgono, ad ogni passo, il lato che rende il peso condizionatamente migliore. La derandomizzazione formale fornisce la *giustificazione teorica* del perché una regola greedy così semplice funzioni e preservi il rapporto 2.

3 Esercizio: Multiway Cut

Problema 3 — Algoritmo di approssimazione per il Multiway Cut

Dato un grafo non orientato $G = (V, E)$, costi $c_e \geq 0$ per ogni arco $e \in E$, e k vertici distinti $s_1, \dots, s_k$, l'obiettivo è rimuovere un insieme di archi F di **costo minimo** tale che, nel grafo residuo $(V, E \setminus F)$, nessuna coppia s_i, s_j (con $i \neq j$) si trovi nella stessa componente connessa.

Caso base: per $k = 2$ il problema coincide con il *min-cut* (separare due soli terminali), risolvibile in tempo polinomiale tramite max-flow (teorema di max-flow/min-cut).

Algoritmo proposto per k generico: per $i = 1, \dots, k$, sia F_i il min-cut che separa s_i da tutti gli altri terminali $\{s_1, \dots, s_k\} \setminus \{s_i\}$ (cioè si accorpano idealmente $s_1, \dots, s_{i-1}, s_{i+1}, \dots, s_k$ in un unico super-nodo e si calcola il min-cut fra s_i e quel super-nodo). La soluzione proposta è

$$F = \bigcup_{i=1}^k F_i.$$

Da dimostrare: F è una 2-approssimazione della soluzione ottima.

Traccia di soluzione ragionata

1) Ammissibilità. F separa certamente tutte le coppie, perché per ogni coppia (s_i, s_j) , l'insieme F_i da solo già disconnette s_i da tutti gli altri terminali (incluso s_j); quindi dopo aver rimosso $F \supseteq F_i$, s_i e s_j non possono più essere connessi. Questo è dato per assunto nel testo dell'esercizio ("obviously a feasible solution").

2) Idea per il rapporto 2 — ragionamento sui tagli ottimi locali

Sia F^* la soluzione ottima del multiway cut, di costo OPT. L'idea chiave è la seguente:

- F^* , rimosso dal grafo, spezza V in (almeno) k componenti, una per ciascun terminale s_i (eventualmente altre componenti senza terminali, che possiamo attaccare a piacere ad una qualsiasi componente terminale senza aumentare il costo). Sia D_i^* la componente di F^* contenente s_i .
- Il taglio che separa D_i^* dal resto del grafo (cioè separa s_i da tutti gli altri terminali) è **ammissibile** per il sotto-problema "separa s_i dagli altri $k - 1$ terminali", quindi il suo costo è $\geq$ il costo del min-cut F_i usato nell'algoritmo:

$$c(F_i) \leq c(\partial D_i^*),$$

dove ∂D_i^* denota gli archi di F^* sulla frontiera di D_i^* .

- Ogni arco di F^* sta sulla frontiera di **al più due** componenti D_i^*, D_j^* (un arco ha solo 2

estremi, quindi tocca al più 2 componenti distinte). Perciò, sommando su tutti gli i :

$$\sum_{i=1}^k c(\partial D_i^*) \leq 2c(F^*) = 2 \text{OPT}.$$

3) Conclusione. Combinando i due punti precedenti,

$$c(F) \leq \sum_{i=1}^k c(F_i) \leq \sum_{i=1}^k c(\partial D_i^*) \leq 2 \text{OPT}.$$

(La prima disuguaglianza è la subadditività del costo rispetto all'unione insiemistica: $c(\bigcup_i F_i) \leq \sum_i c(F_i)$, poiché gli insiemi possono sovrapporsi ma i costi sono non negativi.) Dunque l'algoritmo che unisce i k min-cut locali produce una soluzione di costo al più 2OPT : è una **2-approssimazione**.

Esempio

Grafo a “stella” con centro c e $k = 3$ terminali s_1, s_2, s_3 collegati direttamente a c con costi 1, 1, 1 (nessun altro arco). Ottimo: basta tagliare i 3 archi intorno a c ? In realtà basta isolare c da due dei tre terminali e lasciarlo unito al terzo: $\text{OPT} = 2$ (si tagliano 2 dei 3 archi, lasciando c nella componente di uno dei terminali). L'algoritmo proposto calcolerebbe, per ciascun i , il min-cut che isola s_i da tutto il resto: in questo grafo, isolare s_i costa esattamente 1 (basta tagliare l'arco (s_i, c)). Quindi $F = F_1 \cup F_2 \cup F_3$ taglia tutti e 3 gli archi, con costo 3. Il rapporto ottenuto qui è $3/2 \leq 2$, coerente col bound dimostrato (che è un *caso peggiore* nel senso di garanzia superiore, non un valore raggiunto sempre esattamente).

4 Riepilogo dei risultati

Problema / algoritmo	Rapporto	Idea principale
Max-SAT, moneta equa (Johnson)	2	Ogni clausola soddisfatta con prob. $\geq 1/2$
Max-SAT, Yannakakis	$4/3$	Tratta separatamente le clausole con 1 letterale
Max-SAT, LP + arrotondamento (G&W)	$e/(e-1) \approx 1.58$	Arrotondamento secondo x_j^* della soluzione LP
Max-SAT, Johnson + G&W combinati	$4/3$	Media dei due bound, mai sotto $3/4$
Max-Cut, moneta equa	2	Ogni arco tagliato con prob. $1/2$
Max-Cut, SDP (Goemans-Williamson)	$\approx 1/0.8785$	Bound su OPT via progr. quadratica
Max-Cut, derandomizzato greedy	/ 2	Aspettative condizionate, scelta locale ottima
Multiway Cut, unione di min-cut	2	Ogni arco ottimo conta al più 2 volte

Filo conduttore da ricordare per l'esame

- Lo schema generale è sempre: *limite superiore su OPT* (banale, oppure via LP/SDP) + *algoritmo (randomizzato)* + *stima del valore atteso* $\geq$ frazione dell'upper bound.

- Un algoritmo randomizzato con garanzia sul valore atteso può spesso essere reso deterministico (*derandomizzazione*), tipicamente con il metodo delle aspettative condizionate: si fissano le variabili una alla volta, scegliendo sempre l'opzione che non fa scendere il valore atteso condizionato.
- Combinare due algoritmi con punti deboli complementari (Johnson + G&W) è una tecnica generale per migliorare il rapporto di approssimazione nel caso peggiore.